

UNIT - II

Top-Down Design with Functions: Building Programs from Existing Information, Library Functions, Top-Down Design and Structure Charts, Functions without Arguments, Functions with Input Arguments.

Pointers and Modular Programming: Pointers and the Indirection Operator, Functions with Output Parameters, Formal Output Parameters as Actual Arguments.

Top-Down Design with Functions

- Top-Down Design is a programming approach where we start with a large problem and break it into smaller, manageable sub-problems.
- Each sub-problem is implemented using a function.
- A Structure Chart is a diagram that shows the relationship between the main module and its sub-modules.
- The big task "Food Order System" is divided into smaller tasks, and each

TOP - DOWN

Module A
(Main Module)

```
graph TD; A["Module A  
(Main Module)"] --> B["Module B"]; A --> C["Module C"]; A --> D["Module D"]; A --> E["Module E"];
```

Module B

Module C

Module D

Module E

TOP - DOWN

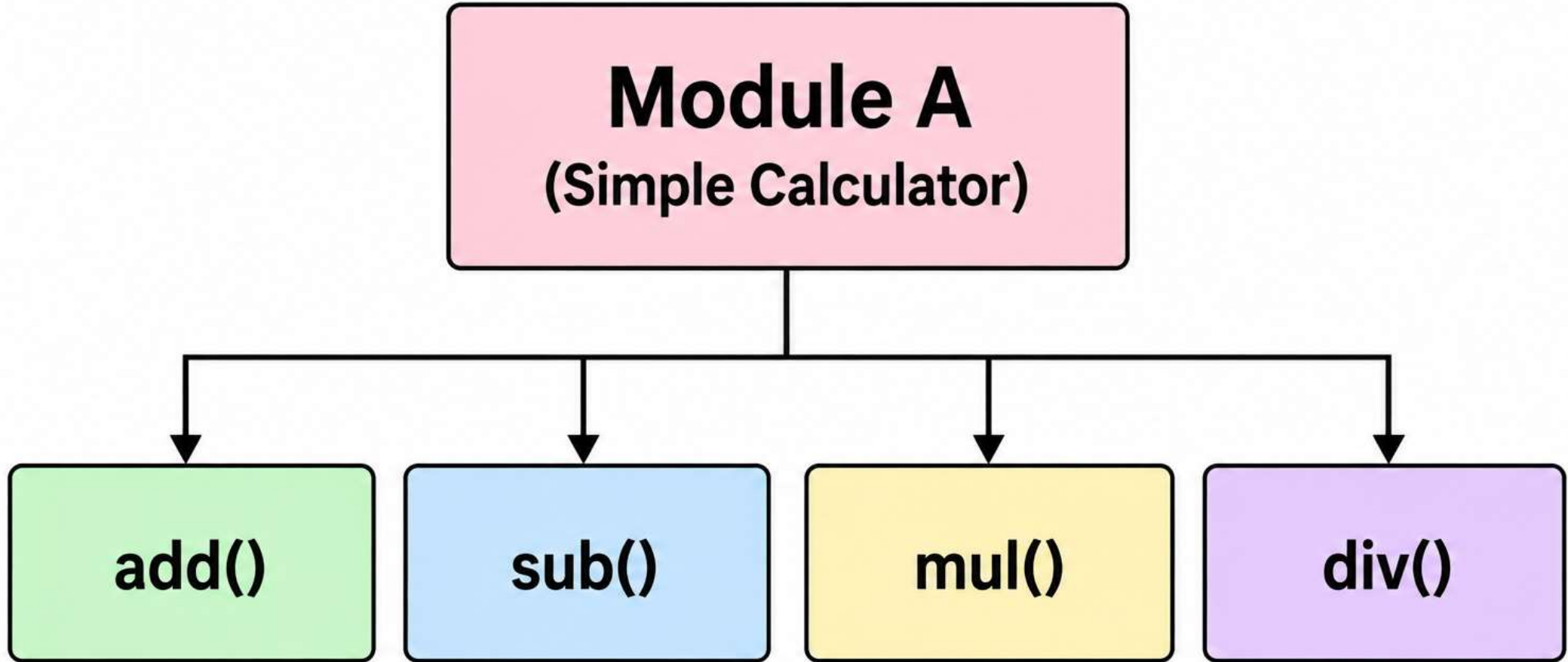
Module A
(Simple Calculator)

add()

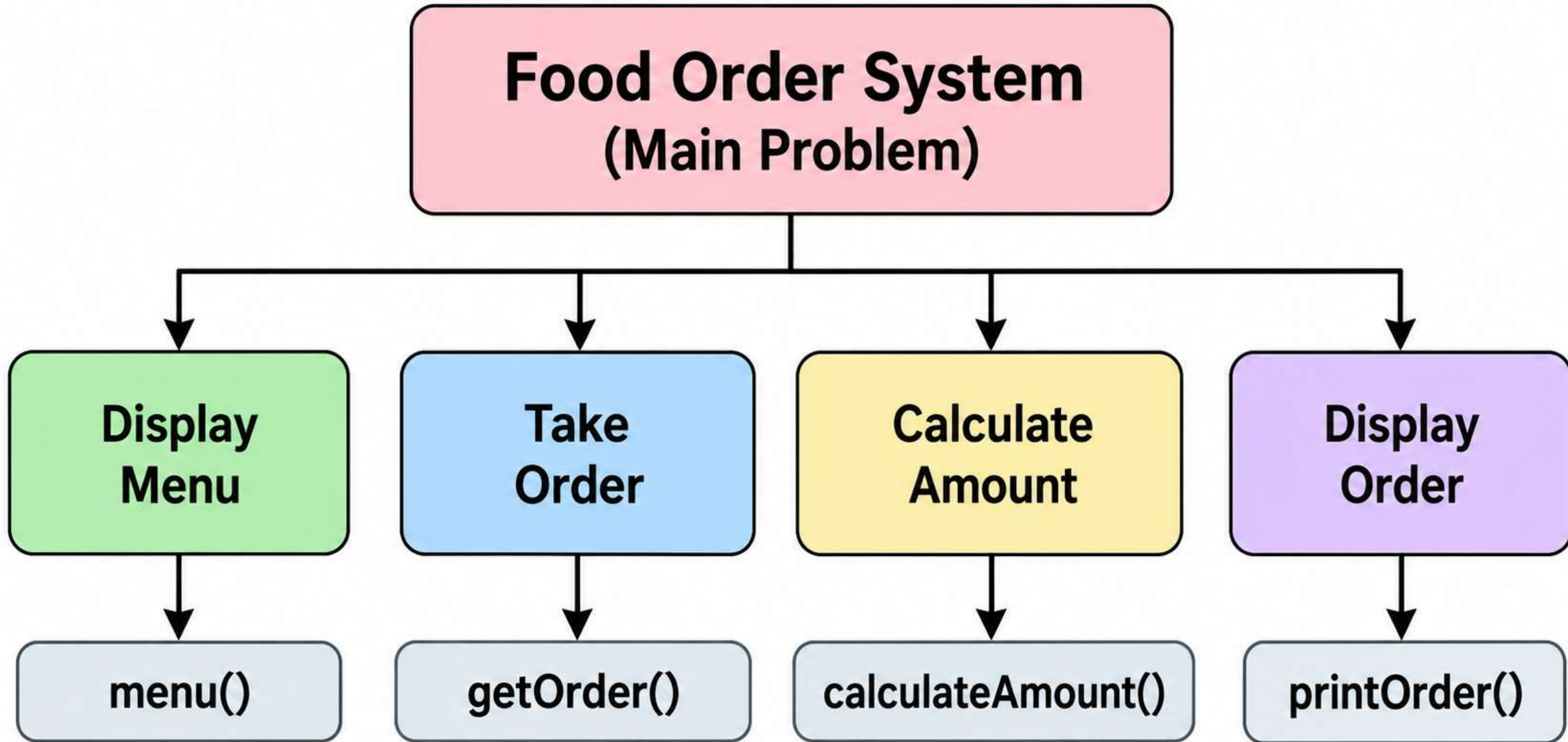
sub()

mul()

div()



1. Top-Down Design

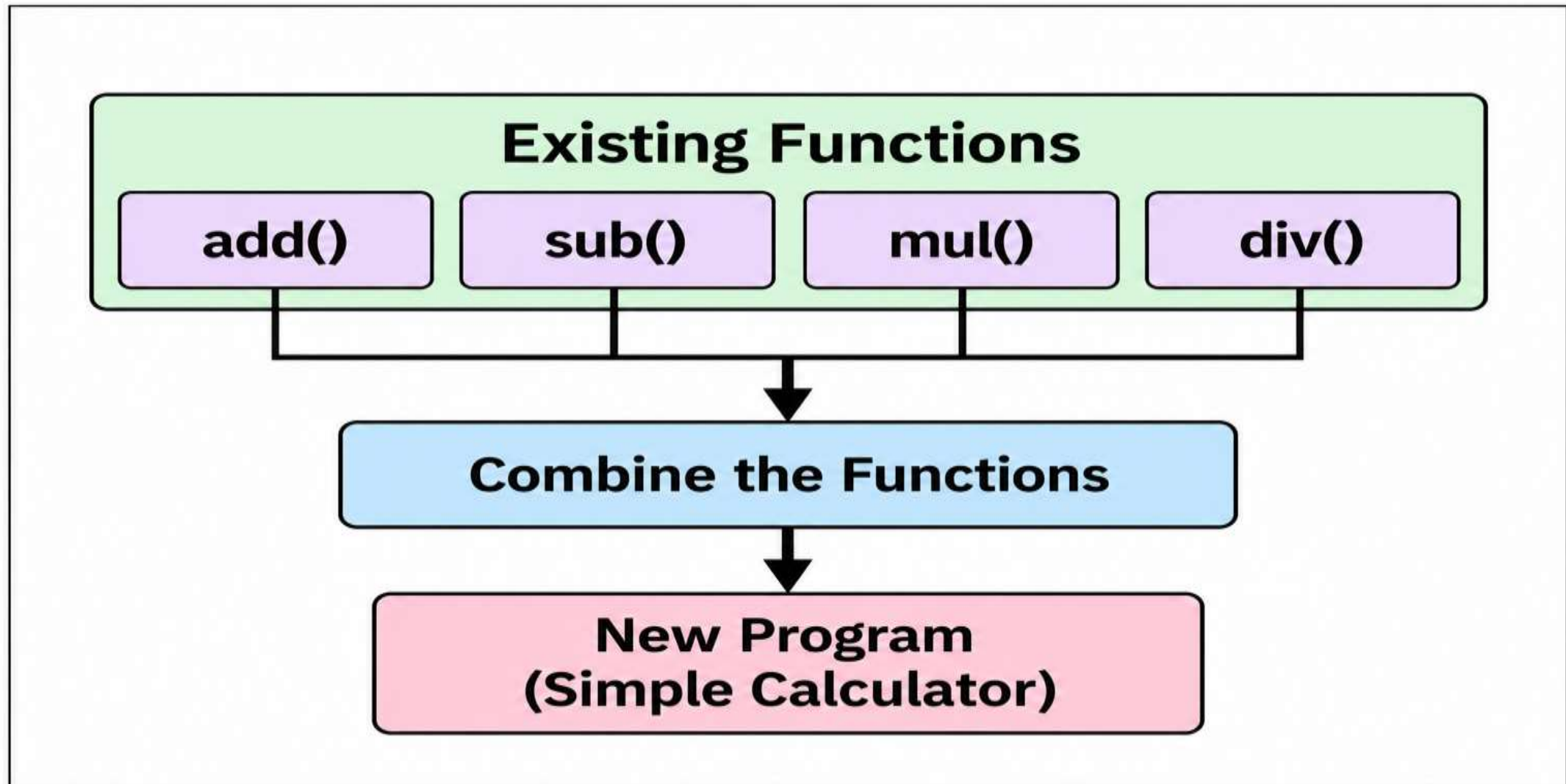


Building Programs from Existing Information

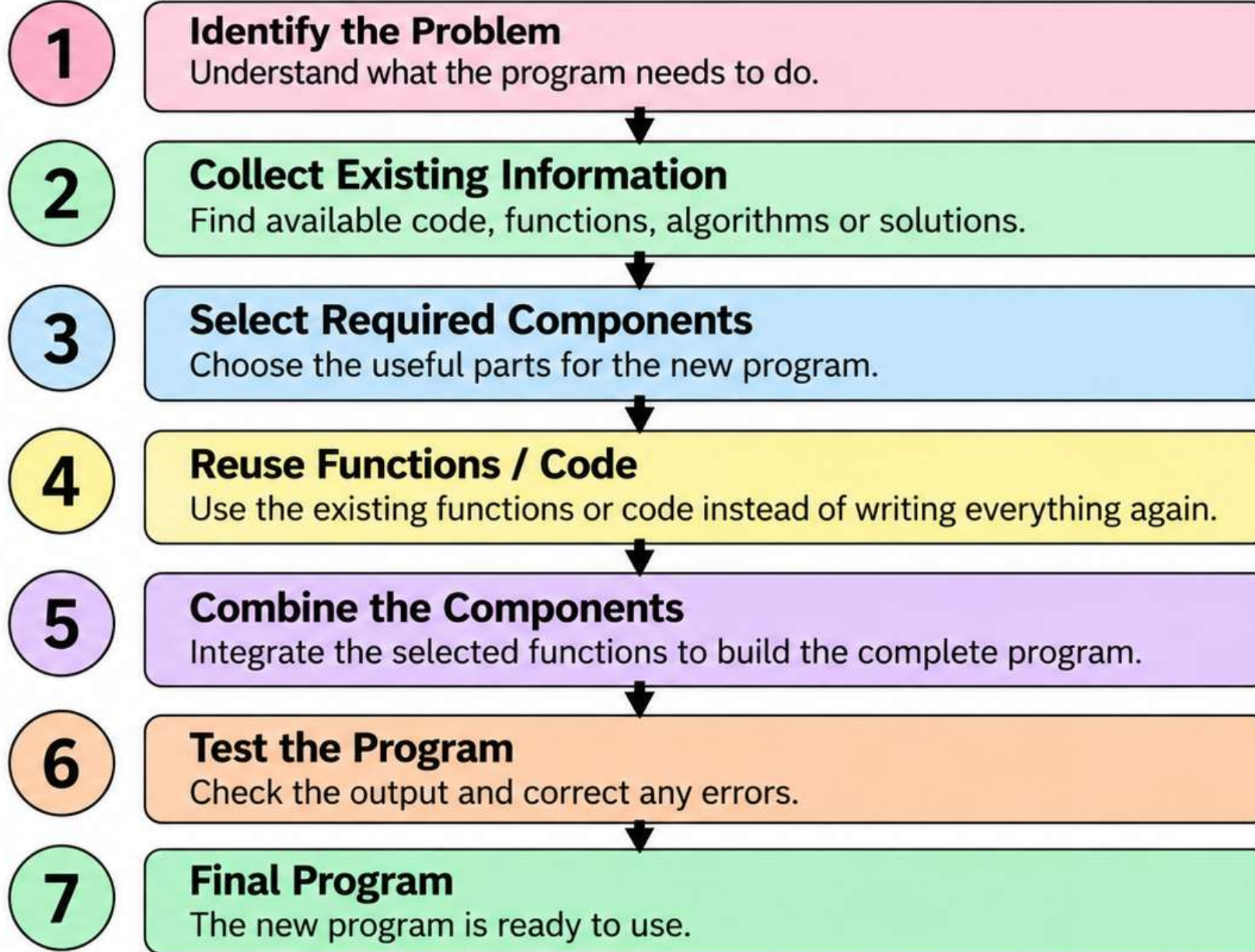
- Using information that is already available.
- Reusing existing functions or code.
- Combining known solutions to create a new program.
- Avoiding unnecessary repetition of code.

Existing Information → Reuse → New Program

Building Programs from Existing



Steps for Building Programs from Existing Information



Why Reuse Existing Information

- Saves programming time.
- Reduces code duplication.
- Makes programs easier to develop.
- Reduces errors.
- Makes programs easier to understand.
- Encourages code reusability.

Functions

- A function is a group of statements written together to perform a specific task.
- In C, we can divide a large program into a number of subprograms, and each subprogram is called a function.
- The function contains a set of programming statements enclosed within { }.
- A function can be called multiple times to provide reusability and modularity to a C program.
- In other words, we can say that the collection of functions creates a program.

Types of functions

There are two types of functions in C programming:

1. Library Functions: are the functions which are declared in the C header files such as scanf(), printf(), gets(), puts(), ceil(), floor() etc.

2. User-defined functions: are the functions which are created by the C programmer, so that programmer can use it many times.

Elements of User-defined function

The user-defined function in C can be divided into three parts:

1. Function declaration (or) prototype of a function
2. Function call (or) signature of a function (or) calling a function
3. Function definition (or) Function body (or) called function

1.Function declaration

A function must be declared globally in a program to tell the compiler about the function name, function parameters, and return type.

Syntax:

`return_type function_name (argument list);`

Example:

```
void add( int x,int y );
```

2.Function call

Function can be called anywhere in the program.

Syntax:

`function_name (argument_list);`

Example:

`add(10,20);`

3. Function definition

It contains the actual statements which are to be executed. It is the most important element to which the control comes when the function is called. Here, we must notice that only one value can be returned from the function.

Syntax:

```
return type function name (argument list)
{
    function body;
}
```

//Program for User-defined functions

```
#include<stdio.h>
int x=10,y=20;
void add(int x,int y);
void main()
{
    add(x,y);
}
void add(int x,int y)
{
    int c;
    c=x+y;
    printf("The additio is = %d",c);
}
```

Advantages of user-defined function:

- **Modularity**
- **Reusability**
- **Easy Debugging**
- **Improved Readability**
- **Reduced Code Length**
- **Team Collaboration**
- **Ease of Maintenance**

Classification of user-defined function:

- 1. Function with no arguments and no return value**
- 2. Function with arguments and no return value**
- 3. Function with arguments and with return value**
- 4. Function with no arguments and with return value**

1.Function with no arguments and no return value

```
#include<stdio.h>
void add();
void main()
{
    add();
}
void add()
{
    int x,y,z;
    printf("Enter x,y values");
    scanf("%d%d",&x,&y);
    z=x+y;
    printf("The addition is = %d",z);
}
```

2.Function with no arguments and a return value

```
#include<stdio.h>
int add(void);
void main()
{
    int result;
    result=add();
    printf("The addition is = %d",result);
}
int add()
{
    int x,y,z;
    printf("Enter x,y values");
    scanf("%d%d",&x,&y);
    z=x+y;
    return z;
}
```

3.Function with arguments and no return value

```
#include<stdio.h>
int a=10,b=20;
void add(int a,int b);
void main()
{
    add(a,b);
}
void add(int x,int y)
{
    int z;
    z=x+y;
    printf("The addition is = %d",z);
}
```

4.Function with arguments and a return value

```
#include<stdio.h>
void add(void);
void main()
{
    add();
}
void add()
{
    int x,y,z;
    printf("Enter x,y values");
    scanf("%d%d",&x,&y);
    z=x+y;
    printf("The addition is = %d",z);
}
```

Parameter passing mechanism in c

Parameter passing is the mechanism used to pass parameters to a function.

There are 2 types of parameter passing methods.

- 1.Call By Value

- 2.Call By Reference

1.Call By Value or Pass by value

- It is the process of pass the value from calling function to called function.
- In this method, the value of the actual parameters is copied into the formal parameters.
- In this method, we can not modify the value of the actual parameter by the formal parameter. (Original Value is not modified)

```
#include<stdio.h>
void swap(int,int) ; // function declaration
void main()
{
    int num1, num2 ;
    num1 = 10 ;  num2 = 20 ;
    printf("\nBefore swap: num1 = %d, num2 = %d", num1, num2) ;
    swap(num1, num2) ; // calling function
    printf("\nAfter swap: num1 = %d\nnum2 = %d", num1, num2);
}
```

```
void swap(int a, int b) // called function
{
    int temp ;
    temp = a ;
    a = b ;
    b = temp ;
}
```

2.Call By Reference or Pass by Reference

- It is the process of pass the value of the address from calling function to called function.
- In this method, the address of the actual parameters is copied into the formal parameters.
- In this method, we can modify the value of the actual parameter by the formal parameter. (Original Value is modified)

```
#include<stdio.h>
void swap(int *,int *) ; // function declaration
void main()
{
    int num1, num2 ;
    num1 = 10 ;  num2 = 20 ;
    printf("Before swap: num1 = %d, num2 = %d", num1, num2) ;
    swap(&num1, &num2) ; // calling function
    printf("After swap: num1 = %d, num2 = %d", num1, num2);
}
void swap(int *a, int *b) // called function
{
    int temp ;
    temp = *a ;
    *a = *b ;
    *b = temp ;
}
```